Voice Bot — Confirmed Problems To Fix

1. Missing API key crashes with a raw error

If the API key isn't set when the bot starts, it does fail immediately, which is good, but it shows a raw, unhandled Python error (15+ lines of technical traceback) instead of a clear message.

Solution:

Catch this specific error at startup and show a short, clear message instead (e.g. "GROQ_API_KEY is missing - add it to .env") before the raw traceback would otherwise appear.

2. Starting the bot twice shows a misleading "ready" message

If the bot is already running and a second copy is started on the same port, it correctly shows a clear error that the port is already in use - but then keeps going anyway and prints "Bot ready! Open: http://localhost:7860", even though it never actually started.

Solution:

Make the bot actually stop and exit when the port bind fails, instead of continuing on and printing a success message.

3. Saying "mm-hmm" while the bot is talking wrongly counts as an interruption

Real listeners naturally make small sounds like "mm-hmm" or "yeah" without meaning to interrupt. A short acknowledgment sound was fully picked up and treated as a real interruption and a real conversation turn, cutting the bot off and sending the sound to the AI as if it were an actual question.

Solution:

Add a check so a very short or unclear sound isn't treated as a full interruption - for example, requiring the interrupting speech to be a bit longer or clearer before it's allowed to actually cut the bot off.

4. Interrupting the bot twice in a row, quickly, can fail on the second try

After interrupting the bot once, a second interruption about half a second later did not get picked up at all within a reasonable time window. The bot may not reliably detect being interrupted again while it's still reacting to the first interruption.

Solution:

Needs further investigation into why the second interruption wasn't detected - whether it's a real limitation in how quickly the bot can notice a second interruption, or a timing issue specific to how it was tested. Either way, this needs a closer look before it can be considered safe.

5. A single very short/unclear reply can make the AI say something completely unrelated

Saying just "Yes" with no prior context produced a bizarre, made-up response from the AI (referencing an unrelated song) instead of asking for clarification. The AI guesses wildly when given an ambiguous one-word input instead of admitting it doesn't have enough context.

Solution:
Add an instruction so that when the user's input is too short or unclear to make sense of, the AI asks a clarifying question instead of guessing.

6. Very long sentences can crash the voice output

The text-to-speech engine (Kokoro) has a hard limit on how long a single sentence can be before it errors out completely, instead of just cutting it short. A length limit on the AI's replies was added afterward, which makes this much less likely, but the underlying crash risk is still there if one very long sentence slips through.

Solution:
Split long sentences into smaller pieces before sending them to the voice engine, so a long reply is chopped into safe chunks instead of relying only on keeping the AI's total reply short.

7. A broken voice service could still cause a runaway loop of failed retries

When the previous text-to-speech service (Cartesia) broke, the bot's own "let me apologize out loud" fallback kept trying to speak through the same broken service, failing and retrying itself almost 200 times in about a minute. A basic guard was added afterward (skip the spoken apology if the voice engine itself is what's broken, plus a short cooldown), but it's a simple patch, not a full safety net.

Solution:
Add a proper circuit breaker: after a few failures in a row from the same service, stop trying it automatically for a while instead of relying on a fixed cooldown timer.

8. A second, hidden system silently overrides how long the bot waits for silence

Alongside the simple "how long was it quiet" setting already known about, the bot automatically loads a second, more advanced system ("Smart Turn") that listens to whether what the user said actually SOUNDS finished, and can override the simple setting and make the bot keep waiting even after it's gone quiet. This was loading and running the entire time without anyone knowing it was there, and likely explains some of the harder-to-explain waiting/timing behavior seen during testing.

Solution:
Decide whether to keep this second system on (and tune it deliberately) or turn it off in favor of the simpler, already-understood setting - right now it's on by accident, not by choice.

9. The bot's memory of what it said doesn't match what the user actually heard

When the user interrupted the bot less than a second into a long answer, the bot's memory still recorded the ENTIRE answer as if it had been said in full - not just the part that was actually spoken before being cut off. The memory itself doesn't get corrupted or broken, but it becomes inaccurate: the bot believes it already told the user things the user never actually heard.

Solution:

When the bot gets interrupted, only record what was actually spoken up to that point, not the full reply that was generated.

10. Unclean text does reach the voice engine

Asking the AI to write out numbers "like a big number" produced the literal text "1,2,3,4,5" being sent straight to the voice engine, instead of being spoken out as words. Two similar tests (a list, a website link) happened to come out fine, but this one didn't - showing the AI can't be relied on 100% of the time to keep its own output speech-friendly.

Solution:

Add a dedicated cleanup step between the AI's reply and the voice engine that automatically rewrites things like raw numbers, symbols, and links into a spoken-friendly form, instead of relying only on asking the AI nicely.

11. A single sentence can get chopped into two separate turns mid-way through

Found by accident while testing problem #10 above: a single, continuous sentence with a natural pause in the middle (between two clauses) was split into two separate turns by the bot, and it started generating a reply to the first half before the user had even finished the sentence.

Solution:

Needs further investigation into why the pause was long enough to end the turn early - may be related to problem #8 above (the hidden second waiting-detection system).

12. Short interruption sounds keep getting misheard as Urdu

In two separate tests, short interruption-style phrases ("Mm-hmm", and separately "Wait, stop, never mind") were both misheard as Urdu gibberish instead of the English that was actually said. This seems to be a repeatable pattern, not a one-off - short or abrupt speech during an interruption specifically seems to confuse the system that decides between English and Urdu.

Solution:

Needs further investigation into why short/abrupt speech specifically tends to get misclassified as Urdu, and whether the English-vs-Urdu decision needs adjusting for short utterances.

Relevant, Not Yet Tested

Situations that are plausible and specific to how this bot is built, based on what was found while investigating the confirmed problems above, but haven't been directly tested yet. Not confirmed bugs - included for awareness.

A. The bot may hear itself when not using a browser

When running through a browser (WebRTC), there's built-in echo cancellation that stops the bot from picking up its own voice through the microphone. When running directly through the computer's mic and speakers (no browser), there is no such protection. This is a likely explanation for the original "bot doesn't stop when interrupted" problem, since that was tested in the no-browser mode.

Possible next step:

Test barge-in specifically in the no-browser mode without headphones, to see if the bot's own voice leaking into the mic is causing false or missed interruptions. If confirmed, add an echo-cancellation step, or require headphones for that mode.

B. Every user turn is quietly processed twice

To correctly support both English and Urdu, every single thing the user says is sent to the speech-to-text service twice at once (once assuming English, once assuming Urdu) to see which one makes more sense. This was a deliberate choice, not a mistake, but it means double the cost and double the load on that service for every turn, which is worth being aware of.

Possible next step:

No action needed unless cost or scale becomes a concern - documented here so it isn't mistaken for unexplained resource usage later.